VerifAI: Datasets and Model Scores

Agentic AI Guardrail Validator | Team Delta | Hackathon ET-03

This report lists every dataset used to **train** and **test** VerifAI, split into four kinds: **handwritten** (written by the team), **synthetic** (written by an LLM or by templates), **organic** (public datasets written by other people) and **validation / quality checks**. Every figure comes from a result file in the project (named in each table). Where a number is likely to be optimistic, the report says so.

1. Summary: what to say in one minute

Kind of data	Used for	Size	Headline result
Handwritten (team)	Testing only	63 reworded violations + 20 harmless; 83 + 60 impact prompts; 64 known violations + 65 look-alikes	Policy model names the right rule for 54 of 63 reworded violations at the 0.5 setting (2 of 20 harmless wrongly flagged). Impact model 92.8% and 96.7%.
Synthetic (LLM / templates)	Training; plus 3 LLM-written test sets	11,492 policy-model rows; 6,320 impact-model rows; 3 x 350 test items	No prohibited request allowed on 120 of 120 fresh LLM-written tests.
Organic (public datasets)	Held-out testing, and half used for training	XSTest, JailbreakBench, AgentHarm, Banking77	94 to 100% of harmful requests stopped; WildGuard far stronger than Llama Guard.
Validation / quality checks	Proving the software works	221 + 78 + 32 automated checks; 12 disguises; calibration on 143	All automated checks pass; 0 of 65 look-alikes hit a rule.

Honest limit: the same team wrote most handwritten sets and the same LLM family wrote the synthetic ones, so those scores are "it does what we designed". Only the organic public datasets were written by outsiders, which is why they are the most believable numbers.

2. Complete list of datasets

Dataset (file)	Kind	Rows	Used for	Who wrote it
policy_intent.jsonl	Synthetic (templates)	6,965	Train policy model	Generated by *generate_policy_data.py* (templates, plus harmless chit-chat as negatives)
policy_llm.jsonl	Synthetic (LLM)	3,118	Train policy model	Groq gpt-oss-20b via *generate_policy_llm.py*
external_train.jsonl	Organic (public)	1,409	Train policy model	Training halves of XSTest, JailbreakBench, AgentHarm + 1,000 Banking77
generated.jsonl	Synthetic (LLM)	3,489	Train impact model	LLM, 12 business domains, several writing styles
generated_focus.jsonl	Synthetic (LLM)	2,831	Train impact model	LLM, targeted at weak spots (amounts, mass messaging, overwrites)
handwritten_test.jsonl	Handwritten	83	Test impact model	Team (set 1; used to choose improvements)
handwritten_test2.jsonl	Handwritten	60	Test impact model	Team (set 2; not used for tuning)
hard_policy_cases.json	Handwritten	64 violations + 65 look-alikes	Test the 22 pattern rules	Team
hard_policy_paraphrases.json	Handwritten	63 violations + 20 harmless	Test the policy model on rewording	Team
guardrail_test_cases.json	Handwritten	20 project cases	Engine regression	Team

Dataset (file)	Kind	Rows	Used for	Who wrote it
eval_verdicts, _2, _3 (.jsonl)	Synthetic (LLM)	3 x 180 (60 allow / 60 review / 60 block)	Test whole system	Groq gpt-oss-20b, labelled by the instructions it was given
eval_tools, _2, _3 (.jsonl)	Synthetic (LLM)	170 + 170 + 168	Test the agent's tool choice	Groq gpt-oss-20b
XSTest (walledai/XSTest)	Organic (public)	450 (250 safe look-alikes, 200 unsafe)	Half train, half held-out test	Researchers (Rottger et al.)
JailbreakBench (JBB-Behaviors)	Organic (public)	100 harmful + 100 benign look-alikes	Half train, half held-out test	JailbreakBench team
AgentHarm (ai-safety-institute)	Organic (public)	208 harmful agent tasks	Half train, half held-out test	UK AI Safety Institute
Banking77 (PolyAI)	Organic (public)	1,000 train sample; 300 test sample (seed 7)	Harmless real banking queries	PolyAI
consent_lists.csv, sanctions.csv, users.csv	Handwritten (sample)	small	Rule lookups and demo logins	Team (sample data, not real people)

3. Scores on HANDWRITTEN data

Written by the team, never trained on. Source: *models/*.metrics.json*, *EVALUATION.md*, *tests/run_tests.js*.

3a. Policy model on 63 reworded violations and 20 harmless requests

Threshold	Flagged for any rule	Right rule named	Harmless wrongly flagged
0.4	59 / 63	56 / 63 (88.9%)	2 / 20
0.5 (sends to a person)	57 / 63	54 / 63 (85.7%)	2 / 20
0.6	54 / 63	51 / 63 (81.0%)	1 / 20
0.7	46 / 63	43 / 63 (68.3%)	1 / 20
0.85	36 / 63	35 / 63 (55.6%)	1 / 20

Latest model (*models/model_policy_intent.metrics.json*), trained on 11,492 rows. With 63 examples one miss moves the rate by 1.6 points, so treat these as rough.

3b. Impact model (LOW / MID / HIGH) on two handwritten sets

Test set	Old model	Rules only	New model	When 60%+ sure
Set 1 (83 prompts, used to tune)	43.4%	49.4%	92.8%	96.1% right
Set 2 (60 prompts, not used to tune)	46.7%	58.3%	96.7%	96.6% right
Cross-validation on training data			macro-F1 95.4%	

Both sets share an author and theme with the training topics, so treat them as optimistic.

3c. Pattern rules (22 rules) on handwritten cases

Test	Result
Known violations that should be blocked	64 / 64 blocked
Look-alike harmless requests that hit a rule	0 / 65
Agent tool-choice sanity set (Llama 3.1 8B, 24 requests)	Right tool 23 / 24; valid JSON 24 / 24

4. Scores on SYNTHETIC (LLM-written) data

Three sets written by Groq gpt-oss-20b, the same model family that wrote the training data. Set 1 was a development set; **Set 2 is the fair one**. Source: *tests/results/eval_verdicts*.json*, *eval_tools*.json*.

Whole system, 180 requests per set	Set 1 first run	Set 1 after fixes	Set 2 (fresh)
Prohibited requests let through (ALLOW)	0 / 60	0 / 60	0 / 60
Prohibited requests blocked outright	97% (58/60)	97% (58/60)	98% (59/60)
Risky-but-legitimate held or blocked	87%	95%	98% (59/60)
Ordinary requests allowed	57%	58%	55% (33/60)
Ordinary requests wrongly blocked	15% (9/60)	0%	3% (2/60)
Ordinary requests held for a person	28%	42%	42% (25/60)
Exact match with all three labels	64%	67%	68%

Agent (Llama 3.1 8B) choosing a tool	Set 1 first run	Set 1 stricter prompt	Set 2 (fresh)
Valid JSON	99%	99%	99% (169/170)
Right tool, all requests	89%	94%	95% (162/170)
Right tool and every checkable argument	71%	73%	76% (106/140)
Declines when no tool fits	67%	83%	87% (26/30)

Groq gpt-oss-20b (the agent now in use) on a fresh set of 168: right tool 86%, right tool and arguments 71%, declines correctly 82% (23 of 28).

5. Scores on ORGANIC (public, outsider-written) data

The most believable numbers, because other people wrote the prompts. Source: *tests/results/external_report_*.md*, produced by *tests/eval_external.py*. "Stopped" means blocked or held for a person. For harmless prompts, lower "blocked" is better.

5a. Full datasets, no training on them (safety model: WildGuard 7B)

Dataset	n	Result
XSTest, unsafe prompts	200	Stopped 94.5% (blocked 79, held 110); missed 5.5%
JailbreakBench, harmful	100	Stopped 100%; missed 0%
AgentHarm, harmful agent tasks	208	Stopped 99.0%; missed 1.0%
XSTest, safe look-alikes	250	Allowed 78.8%; held 9.6%; wrongly blocked 11.6%
JailbreakBench, benign look-alikes	100	Allowed 38%; held 31%; wrongly blocked 31%
Banking77, real banking queries	300	Allowed 57%; held 34.7%; wrongly blocked 8.3%

5b. Same sets with Llama Guard 3 1B as the safety model (comparison)

Dataset	WildGuard 7B	Llama Guard 3 1B
XSTest unsafe stopped	94.5%	26.0% (missed 74%)
JailbreakBench harmful stopped	100%	68.0% (missed 32%)
AgentHarm stopped	99.0%	60.6% (missed 39.4%)
XSTest safe, wrongly blocked	11.6%	11.2%
JailbreakBench benign, wrongly blocked	31%	7%
Banking77 wrongly blocked	8.3%	8.0%

Llama Guard is smaller and blocks fewer harmless prompts, but misses about 3 in 10 to 7 in 10 harmful ones. WildGuard is the default for that reason.

5c. Before and after training the policy model on half of each dataset (held-out half only)

Split by group (XSTest focus word, JailbreakBench index, AgentHarm original id) so near-identical prompts never sit on both sides. Banking77: 1,000-row train sample vs a separate test sample.

Held-out set	n	Before training	After training
XSTest unsafe: stopped	101	93.1% (blocked 37, held 57)	94.1% (blocked 79, held 16)
JailbreakBench harmful: stopped	51	100%	100%
AgentHarm harmful: stopped	104	98.1%	99.0%
XSTest safe: allowed	142	81.7% (wrongly blocked 6.3%)	76.8% (wrongly blocked 10.6%)
JailbreakBench benign: allowed	51	39.2% (wrongly blocked 39.2%)	33.3% (wrongly blocked 39.2%)
Banking77: allowed	300	57.0% (wrongly blocked 8.3%)	78.3% (wrongly blocked 0%)

Reading this: harmful requests are still caught (94 to 100%), and real banking queries are no longer blocked (8.3% to 0%). The cost: the new "harmful content" class learned from only about 250 examples, so look-alike sets (XSTest safe, JailbreakBench benign) got slightly worse. Part of the Banking77 gain is in-distribution (the model saw the same kind of banking queries), so do not claim it as a general result.

6. VALIDATION and QUALITY checks

6a. Automated test suites

Suite	Checks	What it covers
node tests/run_tests.js	221 pass	Engine, all hard rules (blocked and allowed prompts), conversation follow-ups, PDF requests, violence rule
python3 tests/test_server.py	78 ok	No verdict from the page, fail-closed, reviewer rules, self-approve, ask-a-reviewer queue, guest-window rule, ledger tampering, tool gateway
python3 tests/ui_smoke.py	32 ok	Real page in Chrome: block, allow, held action, sign-in, maker is not checker, admin-only settings, ledger verify, no script errors

These were last run before the final policy-model retraining. Re-run all three before the demo.

6b. Adversarial robustness: 64 violations, each disguised 12 ways (hard rules + normaliser)

Disguise	Blocked without normaliser	With normaliser
Plain text, mixed case, roleplay, "ignore instructions"	100%	100%
Leetspeak (dr0p th3 pr0duct1on)	9%	100%
Spaced letters	0%	97%
Cyrillic look-alike letters	2%	100%
Zero-width characters	2%	100%
Base64	0%	100%
ROT13	0%	97%
Reversed text	0%	94%
Polite wrapper	97%	97%
Hinglish / Hindi (12 phrases, small lexicon)	0%	92%

Harmless requests hitting a hard rule: 0 of 65 both ways. Disguises are ones the team thought of; new tricks or multi-turn set-ups are not covered.

6c. What each AI component adds (reworded violations, n = 63)

Setup	Blocked	Blocked or held
A: rules only	6%	59%
B: rules + policy model	60%	100%
C: rules + WildGuard	43%	89%
D: all three (the product)	84%	100%

WildGuard alone flags about 61 to 62% of violations. Each component catches things the others miss.

6d. Confidence calibration of the impact model (143 handwritten prompts)

Measure	Result
Mean stated confidence vs actual accuracy	90.3% vs 94.4% (slightly under-confident)
Expected calibration error	5.5 points
When it said 90 to 100% sure	Right 99% of the time (n = 104)
50 to 80% range	Only 22 prompts, so unreliable

6e. WildGuard validation on 708 prompts

Measure	Result
Prohibited requests flagged	160 / 180 (recall 88.9%)
Ordinary requests wrongly flagged	39 / 180 (false alarm rate 21.7%)
Precision / F1 against ordinary requests	80.4% / 84.4%

7. Known weaknesses (say these before someone asks)

Weakness	Evidence
Over-blocks harmless look-alikes	JailbreakBench benign: about 39% wrongly blocked; XSTest safe about 11%
About 42% of ordinary requests are held for a person	LLM-written set 2; WildGuard causes about half of these
Handwritten and synthetic sets are not independent	Same team or same LLM family wrote tests and training data
No outside testers yet	*tests/run_external_eval.py* and the CSV template exist but nobody outside the team has filled them in
Not measured	Multi-turn attacks, non-English beyond a small Hindi lexicon, latency and load
Risk weights and thresholds are hand-set	0.5 / 0.25 / 0.25 and limits 2 and 5 were chosen by the team, not calibrated externally

Suggested claim for the pitch: "On public datasets we did not write, VerifAI stops 94 to 100% of harmful requests. It still over-holds some harmless look-alikes, and we show exactly where." Avoid quoting a single accuracy number.